



SIMsalabim

Blitz and Tricks with SIM Cards

Tomasz Lisowski & Marius Muench

Who are we?



Marius

- Assistant Prof @ University of Birmingham, UK
- Used to play lots of CTF
- Rehosting, Fuzzing, Binary Analysis, Cellular Security

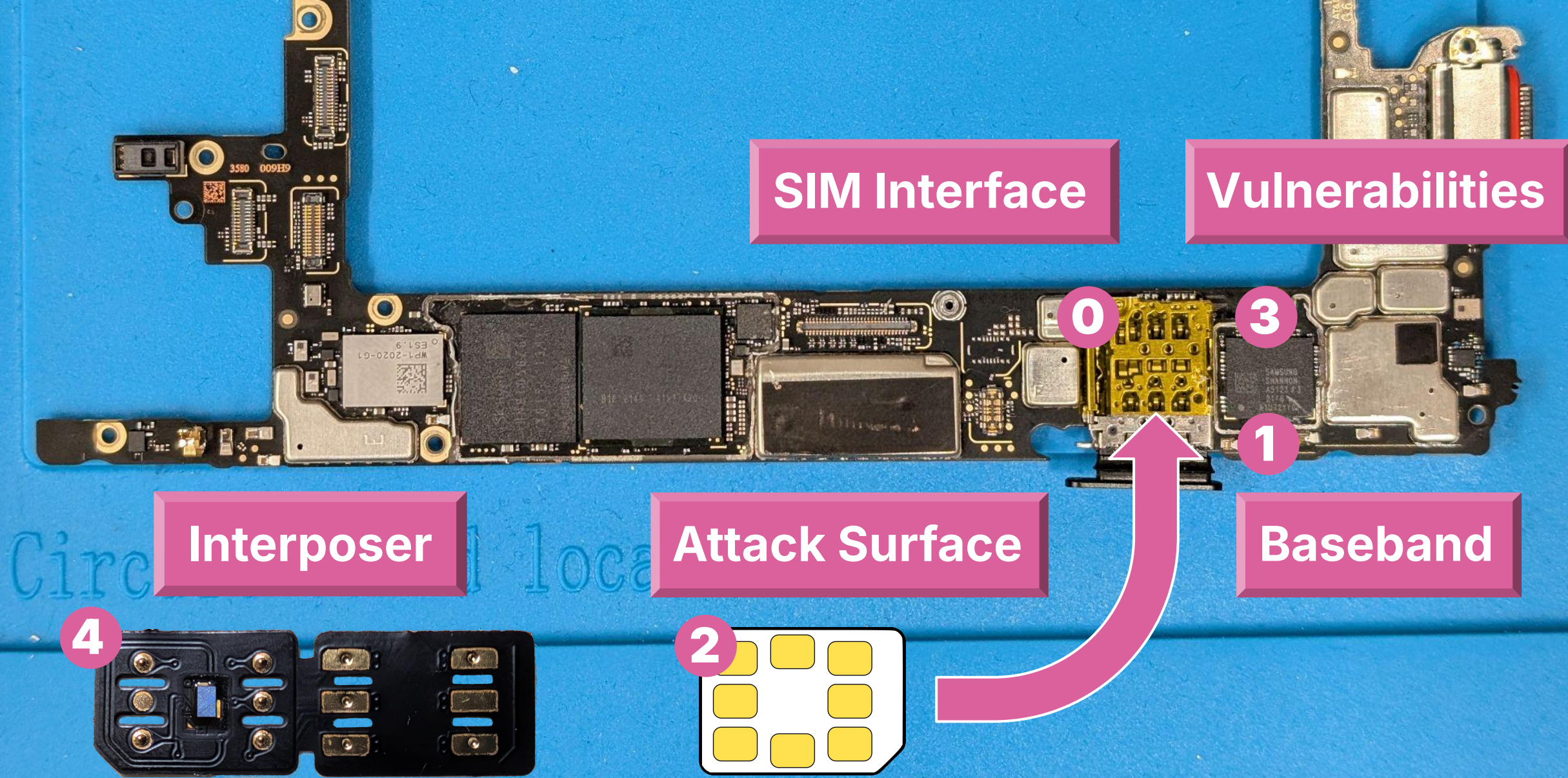
Tomasz

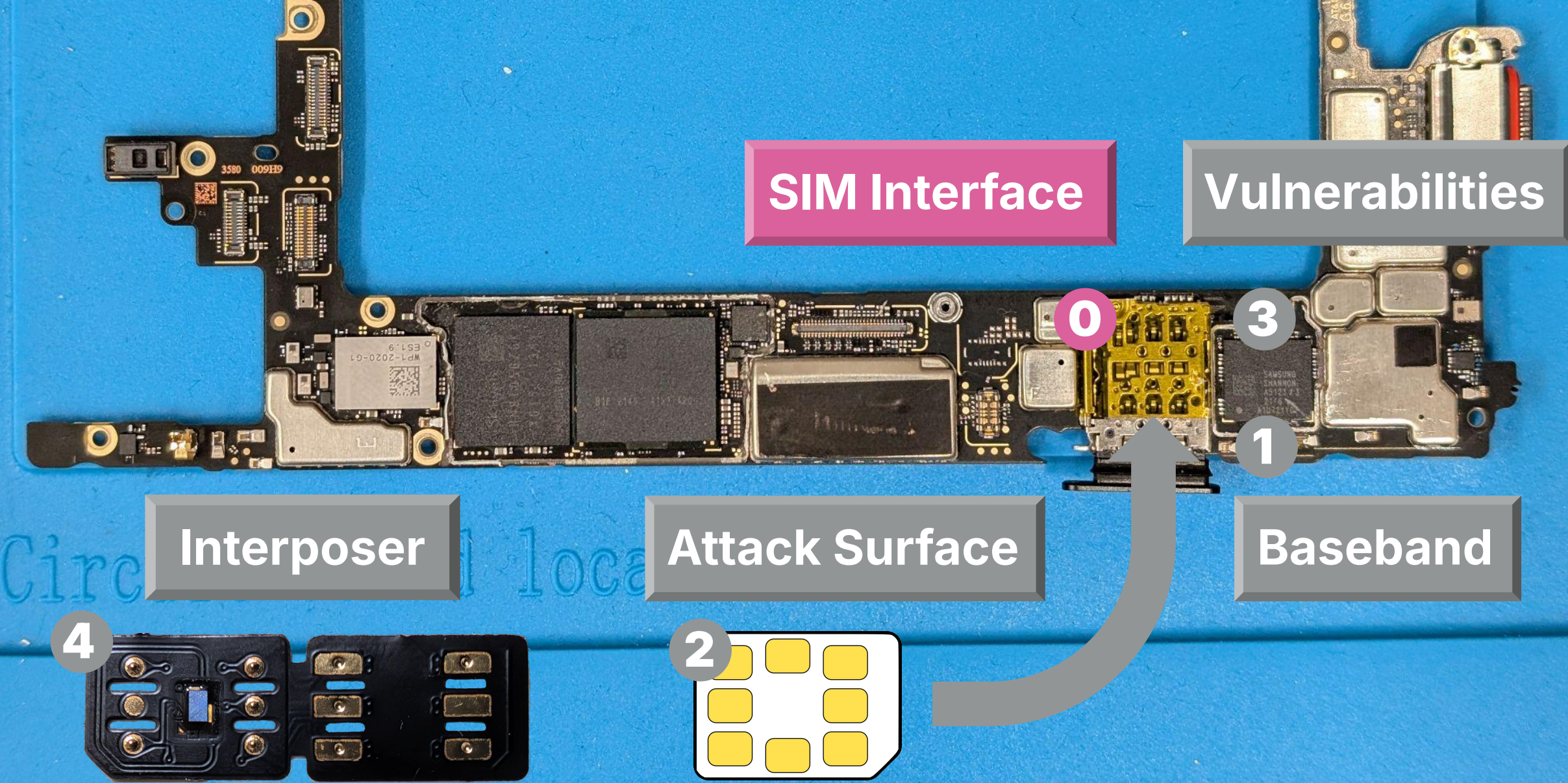
- PhD + Demoscene
- Low level hard/software, Cellular Security



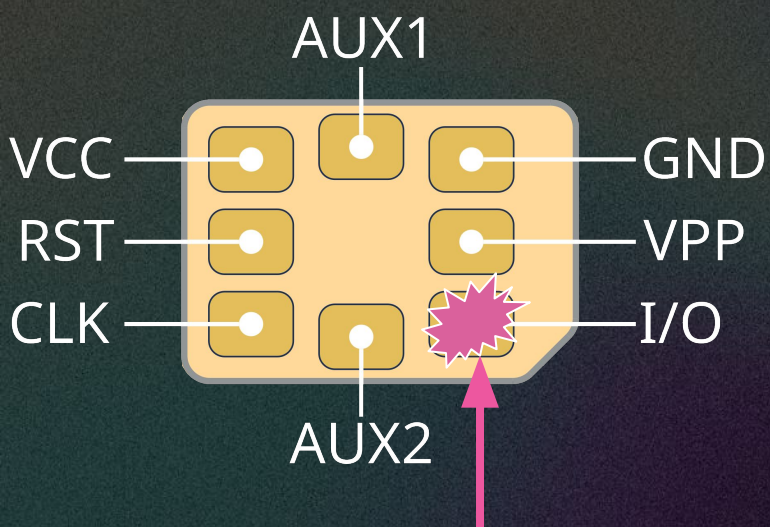
Trick #0

Hacking a phone with a SIM card.

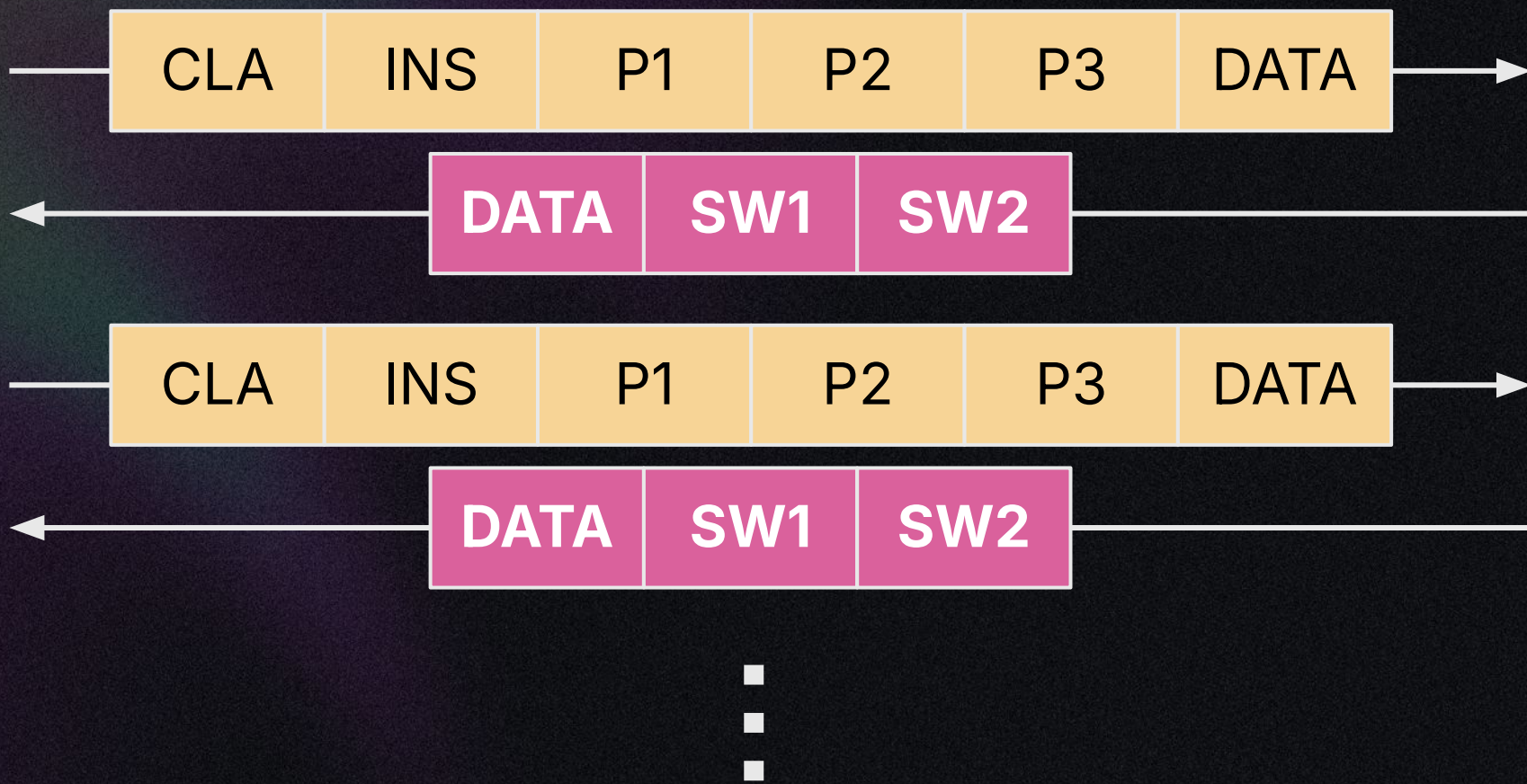




SIM Interface: ISO 7816

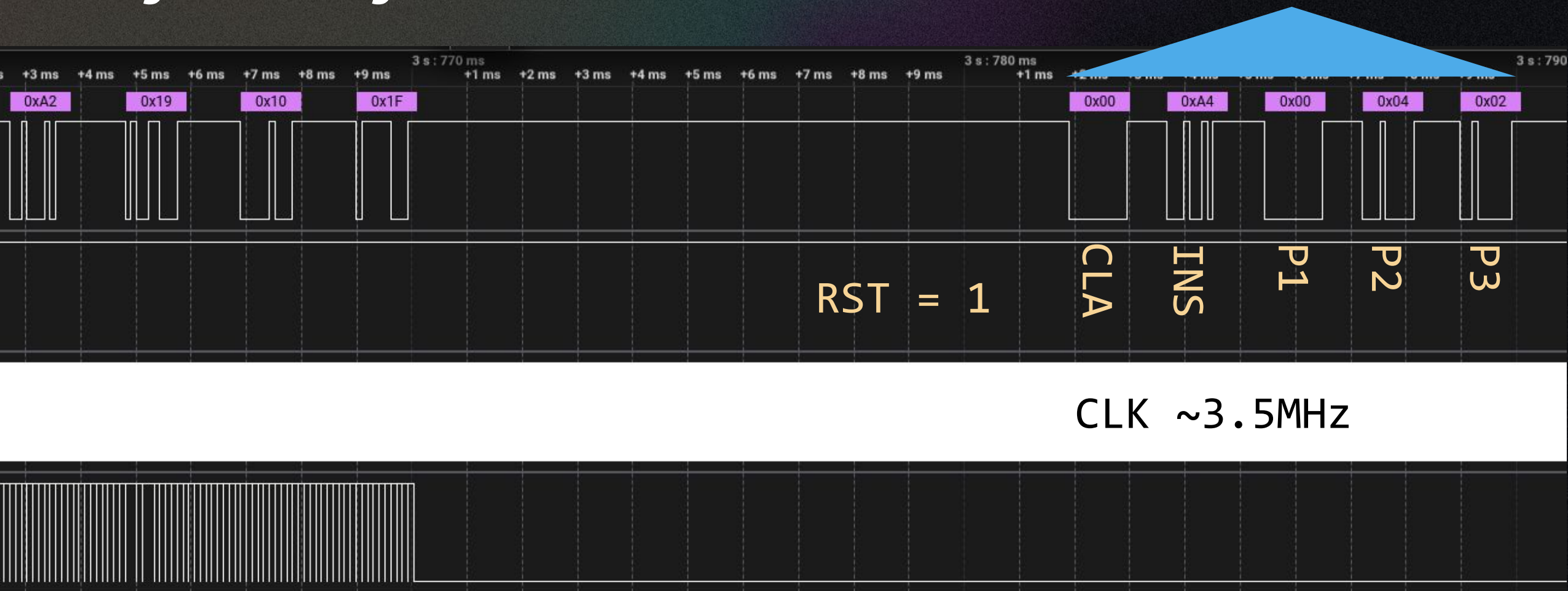


This is our attack
“surface”

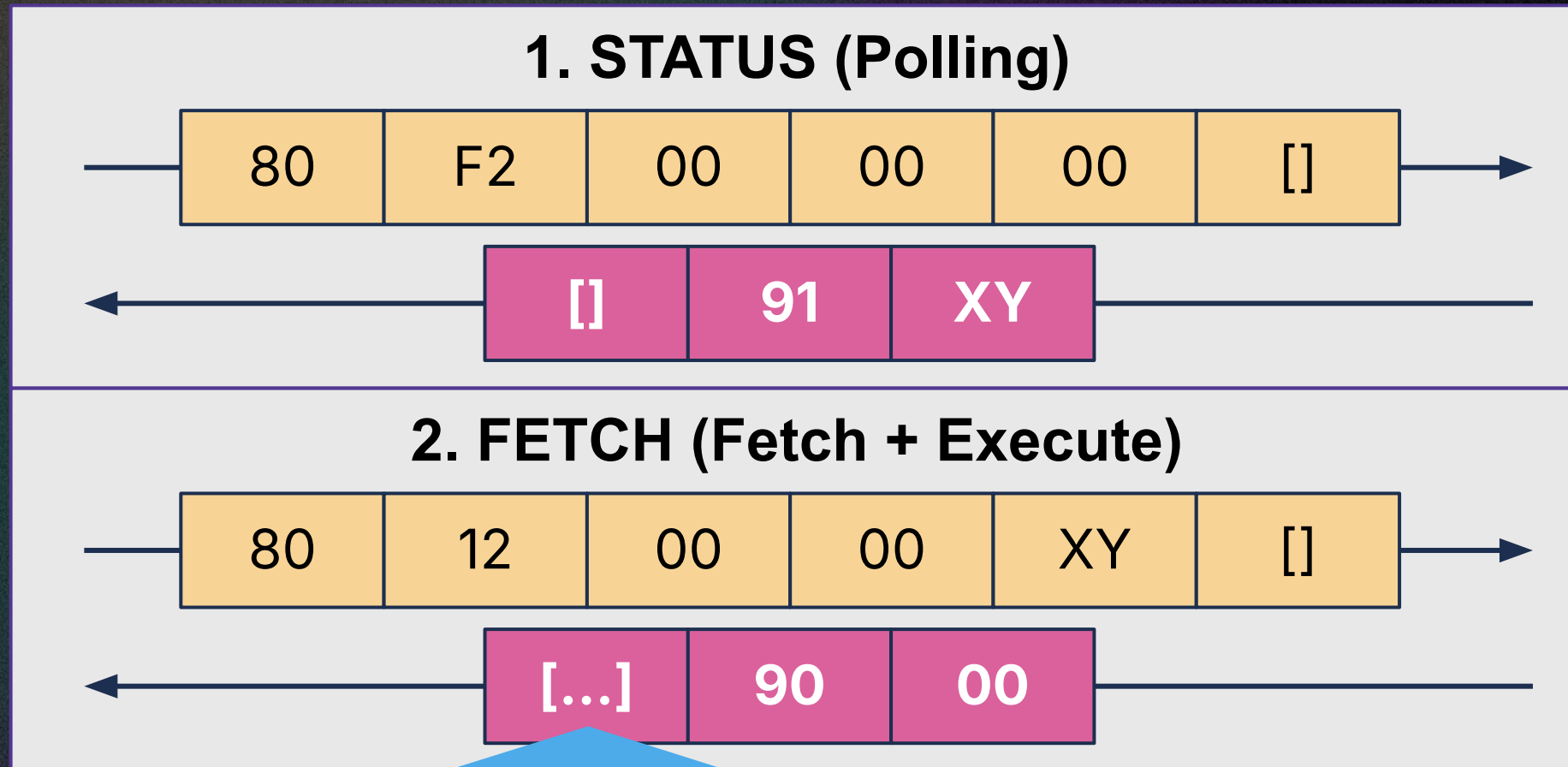


Physical Layer: USART-like

APDU Header



Proactive SIM: SIM Sending Commands to Phone



Proactive Command: Example

— □ ×

```
D0 81 97 81 03 00 25 01 82 02 81 82 85 0A 73 77 53 49 4D 20 4D 65 6E 75 8F 12 01 43
3A 20 4C 41 55 4E 43 48 20 42 52 4F 57 53 45 52 8F 10 02 43 3A 20 44 49 53 50 4C 41
59 20 54 45 58 54 8F 0F 03 43 3A 20 53 45 54 20 55 50 20 4D 45 4E 55 8F 0D 04 43 3A
20 50 4C 41 59 20 54 4F 4E 45 8F 10 05 43 3A 20 4F 50 45 4E 20 43 48 41 4E 4E 45 4C
8F 0F 06 43 3A 20 53 45 54 20 55 50 20 43 41 4C 4C 8F 17 07 53 74 65 61 6C 20 4C 6F
63 61 74 69 6F 6E 20 76 69 61 20 53 4D 53
```

Proactive Command: Example

```
D0 81 97
81 03 00 25 01
82 02 81 82
85 0A 73 77 53 49 4D 20 4D 65 6E 75
8F 12 01 43 3A 20 4C 41 55 4E 43 48 20 42 52 4F 57 53 45 52
8F 10 02 43 3A 20 44 49 53 50 4C 41 59 20 54 45 58 54
8F 0F 03 43 3A 20 53 45 54 20 55 50 20 4D 45 4E 55
8F 0D 04 43 3A 20 50 4C 41 59 20 54 4F 4E 45
8F 10 05 43 3A 20 4F 50 45 4E 20 43 48 41 4E 4E 45 4C
8F 0F 06 43 3A 20 53 45 54 20 55 50 20 43 41 4C 4C
8F 17 07 53 74 65 61 6C 20 4C 6F 63 61 74 69 6F 6E 20 76 69 61 20 53 4D 53
```

Proactive Command: Example

PROACTIVE_UICC_COMMAND

COMMAND_DETAILS #:00 Command:SET_UP_MENU

Qualifier:SELECTION_PREFERENCE_SOFT_KEY|NO_HELP_INFORMATION_AVAILABLE

DEVICE_ID Source:UICC Destination:Terminal

ALPHA_ID AlphaID:"swSIM Menu"

ITEM ItemID:1 ItemTextString:"C: LAUNCH BROWSER"

ITEM ItemID:2 ItemTextString:"C: DISPLAY TEXT"

ITEM ItemID:3 ItemTextString:"C: SET UP MENU"

ITEM ItemID:4 ItemTextString:"C: PLAY TONE"

ITEM ItemID:5 ItemTextString:"C: OPEN CHANNEL"

ITEM ItemID:6 ItemTextString:"C: SET UP CALL"

ITEM ItemID:7 ItemTextString:"Steal Location via SMS"

Trick #0: How it works.

— □ ×

D0	3C	81	03	01	15	00	82	02	81	82	B0	01	00	B1	14	68	74	74	70	73	3A	2F	2F	72	65	2D	76
65	72	73	65	2E	69	6F	2F	85	18	57	61	6E	6E	61	20	73	65	65	20	61	20	6D	61	67	69	63	20
74	72	69	63	6B	3F																						

Trick #0: How it works.

```
D0 3C
81 03 01 15 00
82 02 81 82
B0 01 00
B1 14 68 74 74 70 73 3A 2F 2F 72 65 2D 76 65 72 73 65 2E 69 6F 2F
85 18 57 61 6E 6E 61 20 73 65 65 20 61 20 6D 61 67 69 63 20 74 72 69 63 6B 3F
```

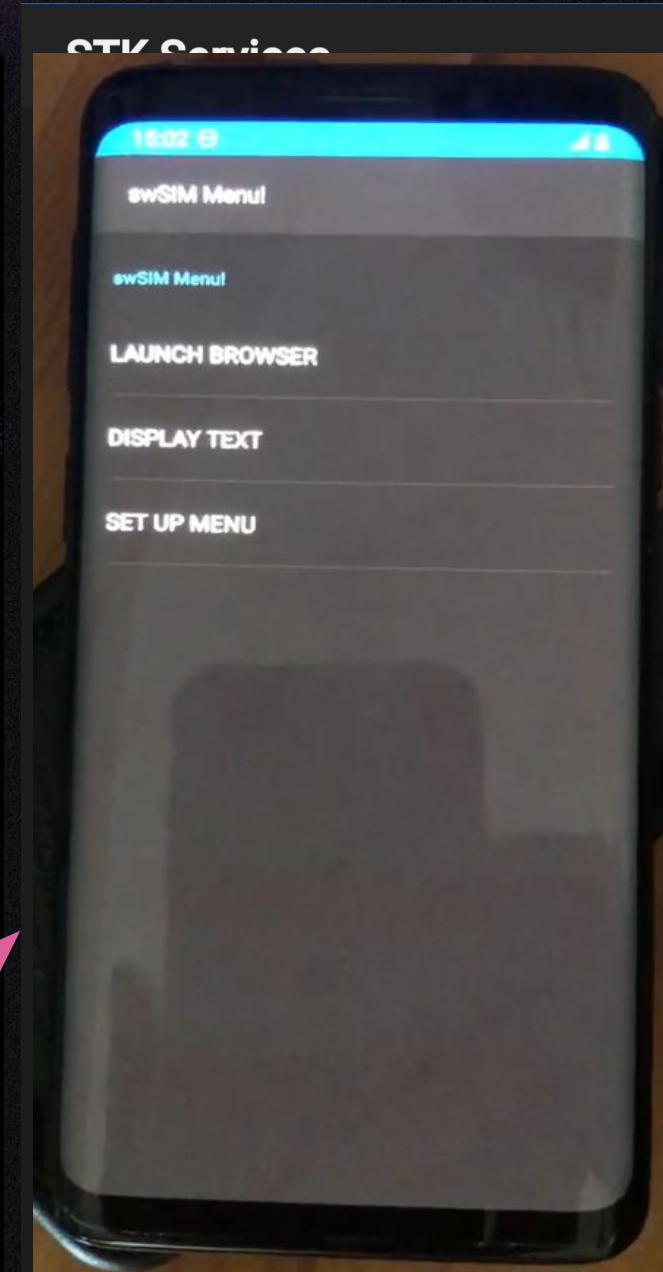
Trick #0: How it works.

```
PROACTIVE_UICC_COMMAND
COMMAND_DETAILS #:01 Command:LAUNCH_BROWSER
  Qualifier:LAUNCH_BROWSER_IF_NOT_ALREADY_LAUNCHED
DEVICE_ID Source:UICC Destination:Terminal
BROWSER_ID Browser_Identity:DEFAULT_BROWSER_SHALL_BE_USED
URL URL:"https://re-verse.io/"
ALPHA_ID AlphaID:"Wanna see a magic trick?"
```

SIM Remote Management

- Operators can:
 - Install apps on SIMs
 - No user consent needed
- Applet capabilities depend on baseband features.
 - TERMINAL PROFILE

YOU CAN ALSO DISPLAY "GRAPHICS"



SIM In

Interposer

Attack Sur

SAMSUNG
SHANNON

4

2

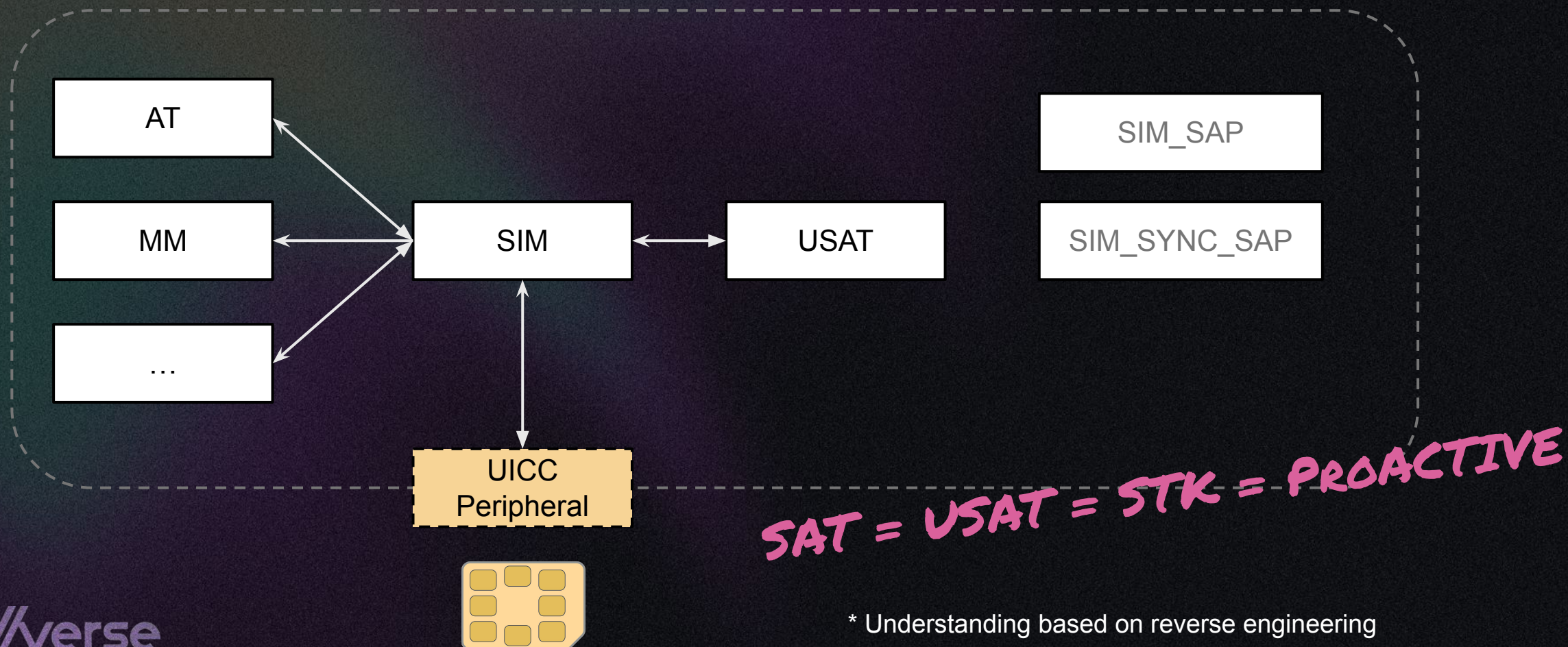
Cellular Baseband Firmware

- Runs on dedicated processor (CP)
- Uses proprietary OS
 - Millions of lines of code
 - Cellular stack split in multiple tasks
- Communicates with:
 - AP
 - Cellular Network
 - SIM

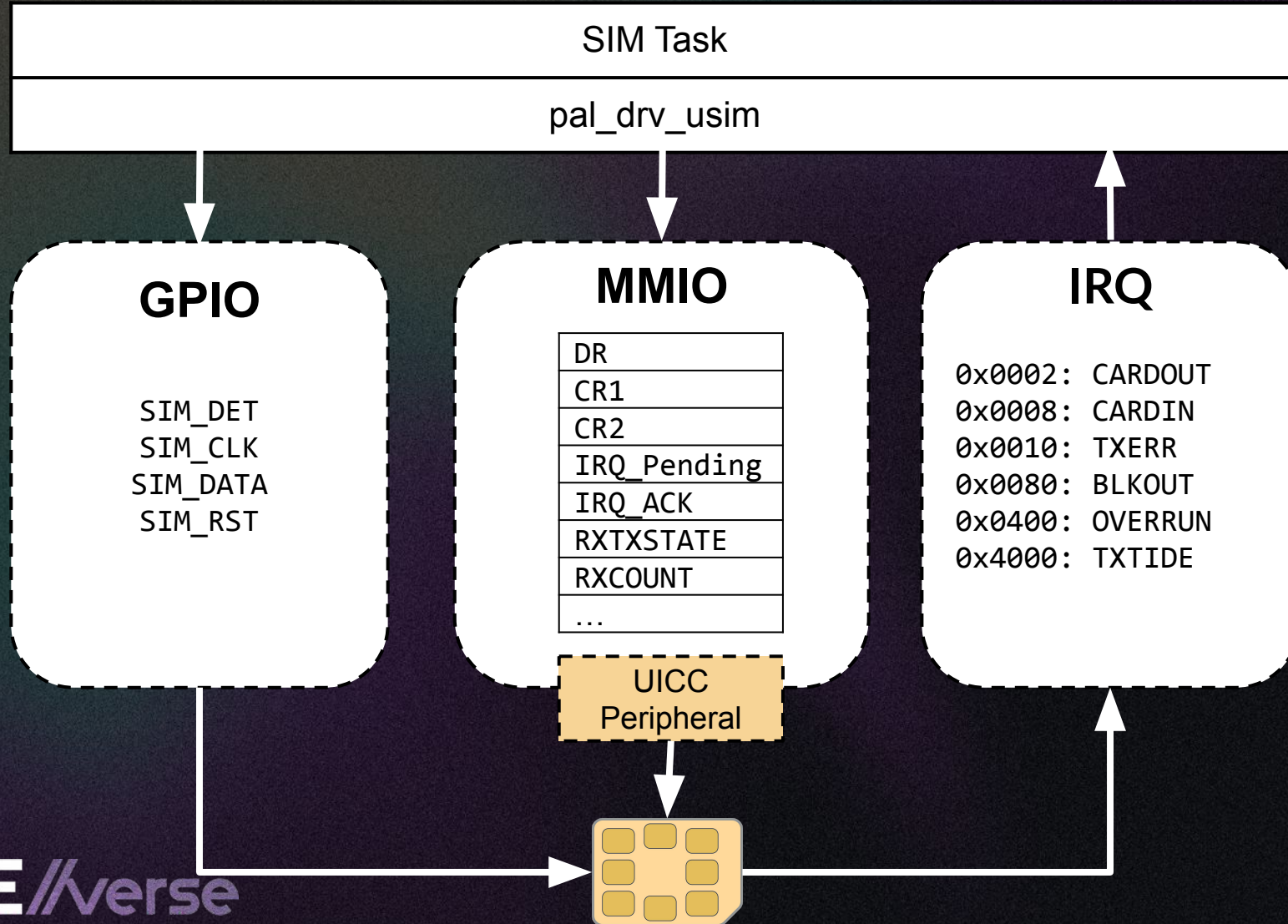


SIM Interaction from the baseband

- Requires cooperation of multiple tasks*:



SIM Task: Low-Level Interaction with UICC



The USAT Task: Proactive Command Parsing

Decoding in `usat_ProcessProactiveCmdReq`

Verifies attributes of command

- Support in terminal profile
- Length

Decoded cmd may be handled:

- Locally on CP
- On AP
- Not at all

```
void usat_ProcessProactiveCmdReq(uint cmd_len, char *cmd) {  
    /* ... */  
    len = verifyCmdLen(cmd, cmd_len);  
    /* ... */  
    proactiveCmdStart = getProactiveStartPtr(cmd, cmd_len);  
    /* ... */  
    CmdType = GetProactiveCmdType(proactiveCmdStart);  
    DisplayProactiveCmdType(CmdType);  
    ret = IsCmdNotAllowed(CmdType);  
    /* ... */  
    ret = usat_DecompileProactiveCmdInd(proactiveCmdStart, len, CmdType);  
    /* ... */  
}
```

Proactive Commands: Handlers in Firmware

```
$ strings modem.bin | grep PROACTIVE_CMD
```

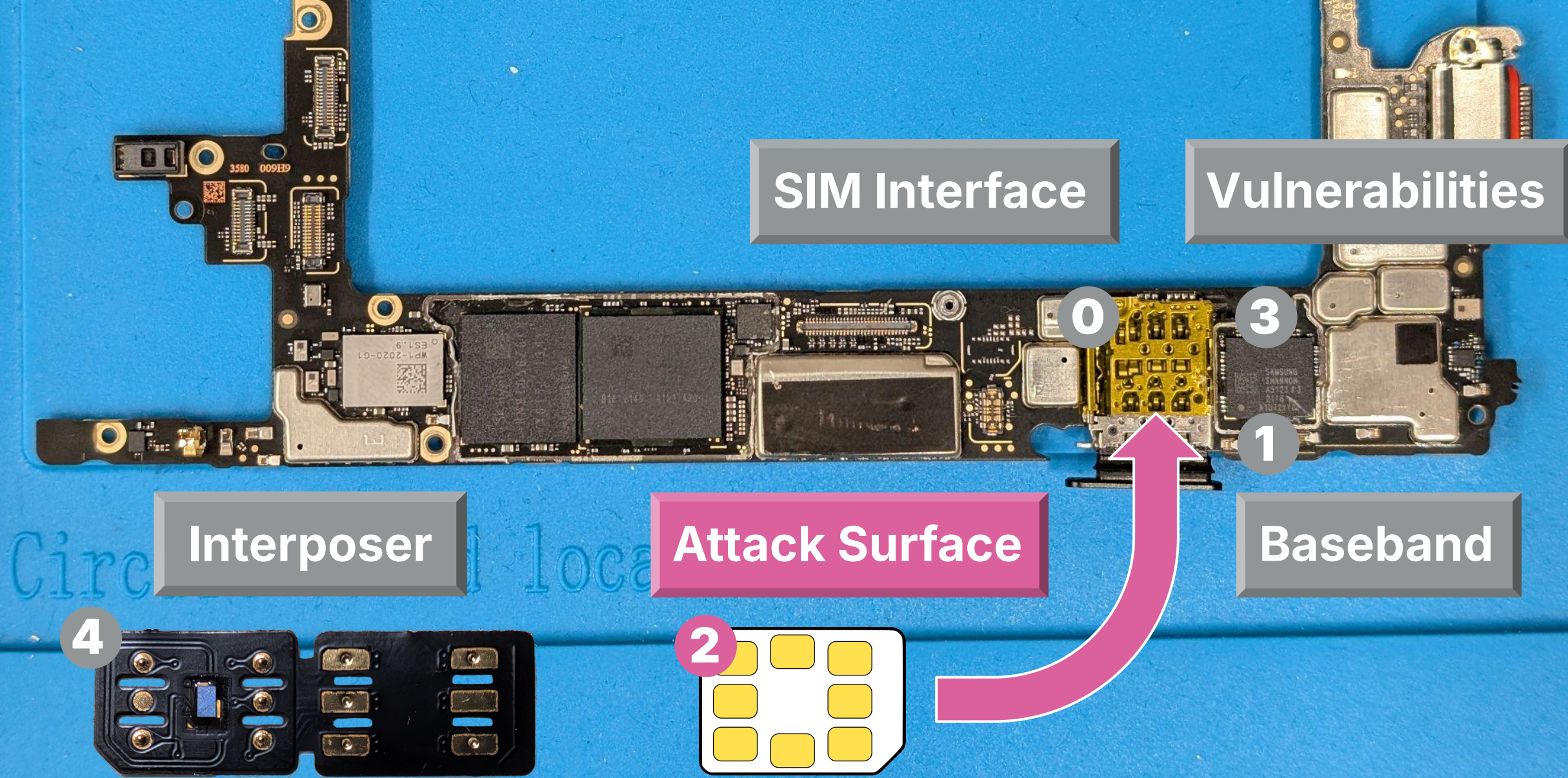
```
PROACTIVE_CMD_DISP_TXT
PROACTIVE_CMD_POLL_INT
PROACTIVE_CMD_POLLING_OFF
PROACTIVE_CMD_PROVIDE_LOCAL_INFO
PROACTIVE_CMD_GET_INPUT
PROACTIVE_CMD_TIMER_MANAGEMENT
PROACTIVE_CMD_SEND_SS
PROACTIVE_CMD_SEND_USSD
PROACTIVE_CMD_SEND_SMS
PROACTIVE_CMD_SEND_DTMF
PROACTIVE_CMD_SETUP_CALL
PROACTIVE_CMD_GET_CHANNEL_STATUS
PROACTIVE_CMD_OPEN_CHANNEL
PROACTIVE_CMD_CLOSE_CHANNEL
PROACTIVE_CMD_RECEIVE_DATA
PROACTIVE_CMD_SEND_DATA
PROACTIVE_CMD_GET_IN_KEY
PROACTIVE_CMD_PLAY_TONE
PROACTIVE_CMD_SETUP_MENU
PROACTIVE_CMD_SELECT_ITEM
PROACTIVE_CMD_SETUP_IDLE_MODE_TEXT
PROACTIVE_CMD_LANGUAGE_NOTIFICATION
PROACTIVE_CMD_LAUNCH_BROWSER
PROACTIVE_CMD_RUN_AT_COMMAND
PROACTIVE_CMD_POWER_OFF_CARD
PROACTIVE_CMD_POWER_ON_CARD
PROACTIVE_CMD_PERFORM_CARD_APDU
PROACTIVE_CMD_GET_READER_STATUS
PROACTIVE_CMD_READ_CARD
PROACTIVE_CMD_SETUP_EVENT_LIST
```

LOT'S OF PARSERS REQUIRED!



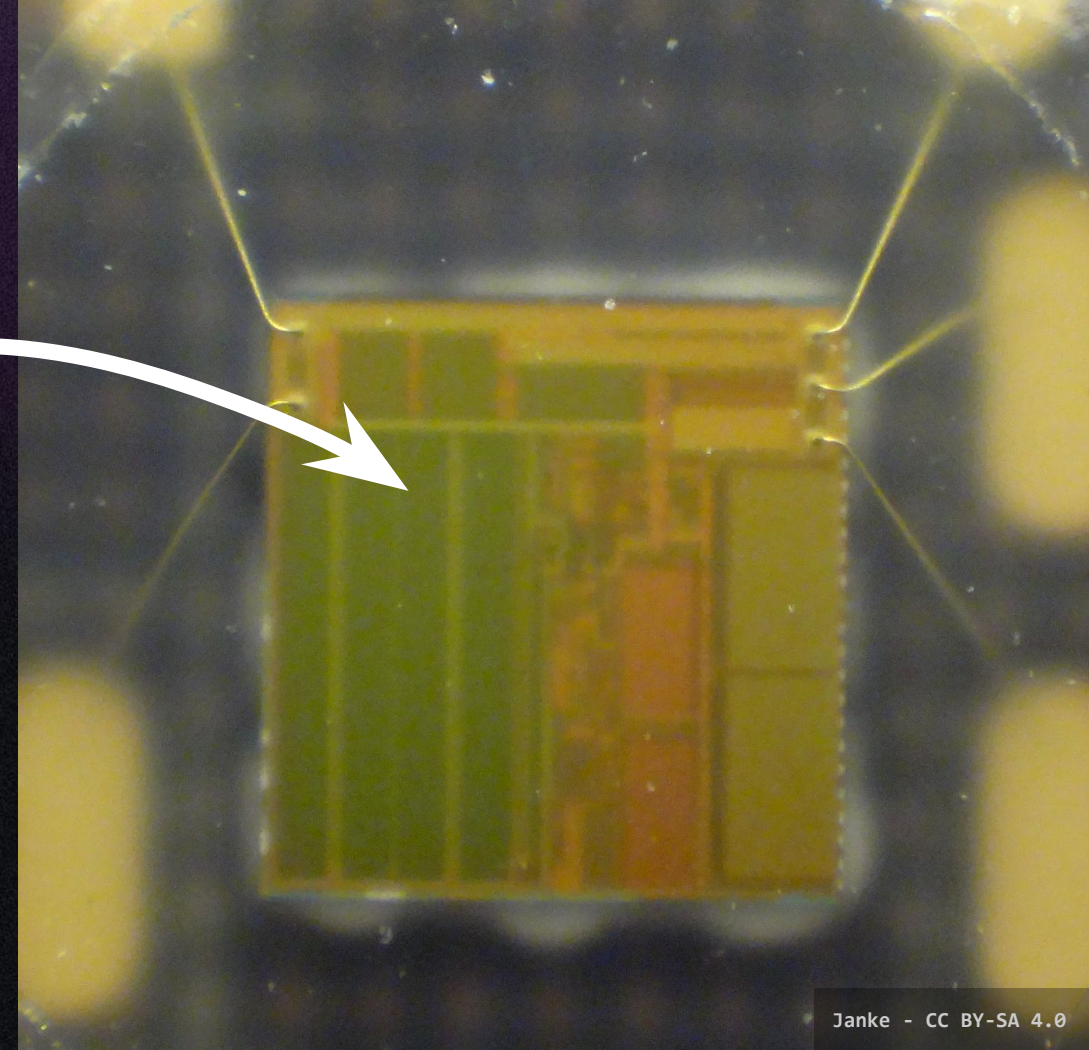
Trick #1

Triggering a bug in the proactive command parser.

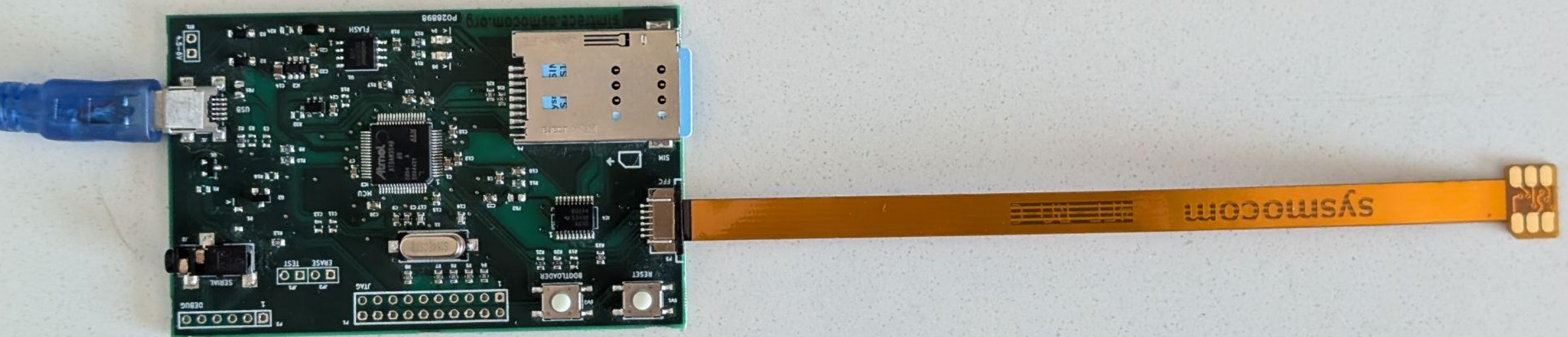


Building Malicious SIMs in Software

- Hardware Required:
 - Java Card
 - Proprietary programmable cards
 - Software & hardware hybrids
 - MitM devices
 - Response rewriting
 - SIMtrace2



SIMtrace2



What are the Challenges?

- Java Cards:
 - Long iteration times
 - Hard to debug (SIM SAT)
 - Cards reject applets
- Our best cards: ~0.2 MIPS
 - Amiga 500 @ ~0.175 MIPS



```
0: iconst_0
1: istore_0
2: iload_0
3: sipush          10000
6: if_icmpge       33
9: iconst_0
10: istore_1
11: iload_1
12: bipush         100
14: if_icmpge       25
17: iload_1
18: iconst_1
19: iadd
20: i2s
21: istore_1
22: goto            11
25: iload_0
26: iconst_1
27: iadd
28: i2s
29: istore_0
30: goto            2
33: return
```

9110002 OPS / ~44.5 SECONDS

Our Solution: SIMurai

Framework for SIM emulation:

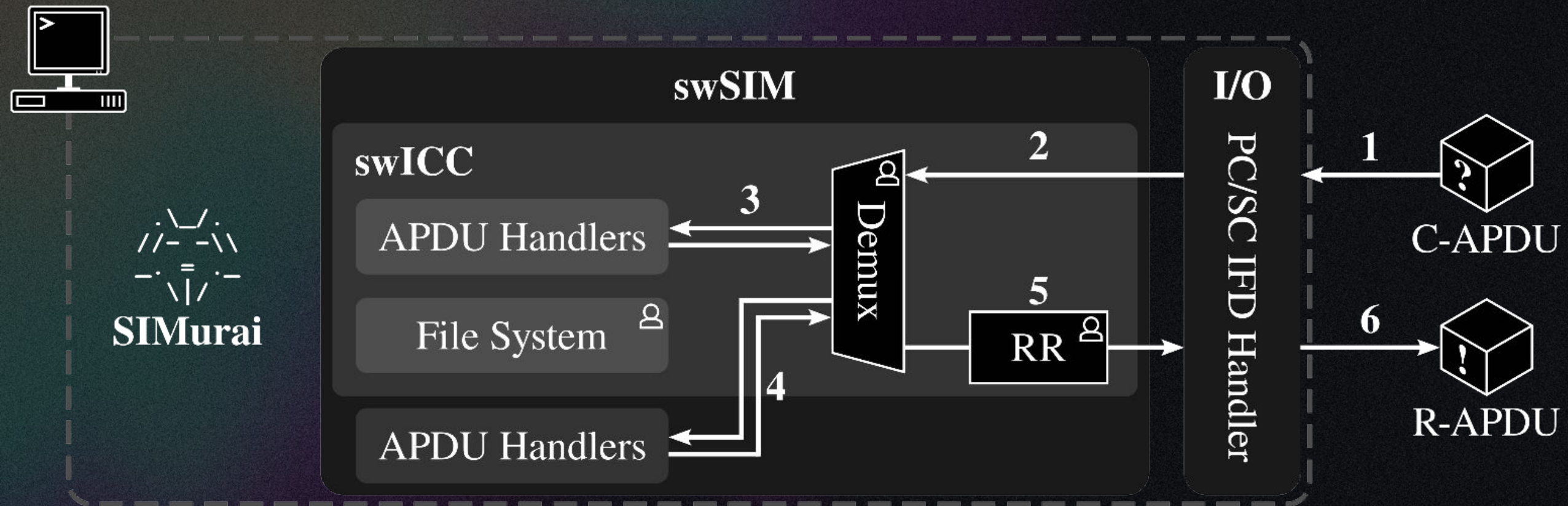
- **swICC** as ICC framework
- **swSIM** as SIM emulator

Designed for research & convenience:

- No more 'reprogramming' cards
- No hardware dependence
- Fast
 - Iteration time
 - Runtime



SIMurai: Under the hood



Let's Go Fuzzing: ME FUZZ

ME FUZZ:

- Built on-top of swSIM
- Connection to COTS phone via SIMtrace2

Results:

- TERMINAL RESPONSE is rarely sent
- Slow
- Rate of execution changes (even for identical commands)
- Sporadic 'timeouts'
- No crashes



: (

FirmWire 🤝 SIMurai

A full-system baseband emulation platform:

- Open source: <https://github.com/FirmWire/FirmWire>
- Support for shannon-based firmware
- Currently no SIM support

Let's connect FirmWare with SIMurai for fuzzing!

- Higher exec/s
- Scalable
- Introspection

Connecting SIMurai

Two Custom PyPeripherals

- GPIO
- USIM

Required steps:

- Mocked USIM state machine
- IRQ support for input delivery
- Connection to SIMurai via PC/SC

```
class USIMPeripheral(LoggingPeripheral):

    def hw_read(self, offset, size):
        offset_name = None
        if offset == OFF_DATA_REGISTER:
            offset_name = "DATA"
            value = self.data_register
            self.shall_read_byte = True
        elif offset == OFF_CR0:
            offset_name = "CR0"
            value = self.cr0
        elif offset == OFF_IRQ_PENDING:
            offset_name = "IRQ_Reason"
            value = self.pending_irqs
        # ...
        else:
            self.log_read(value, size, offset_name)
        return value
```

Connecting SIMurai

Two Custom PyPeripherals

- GPIO
- USIM

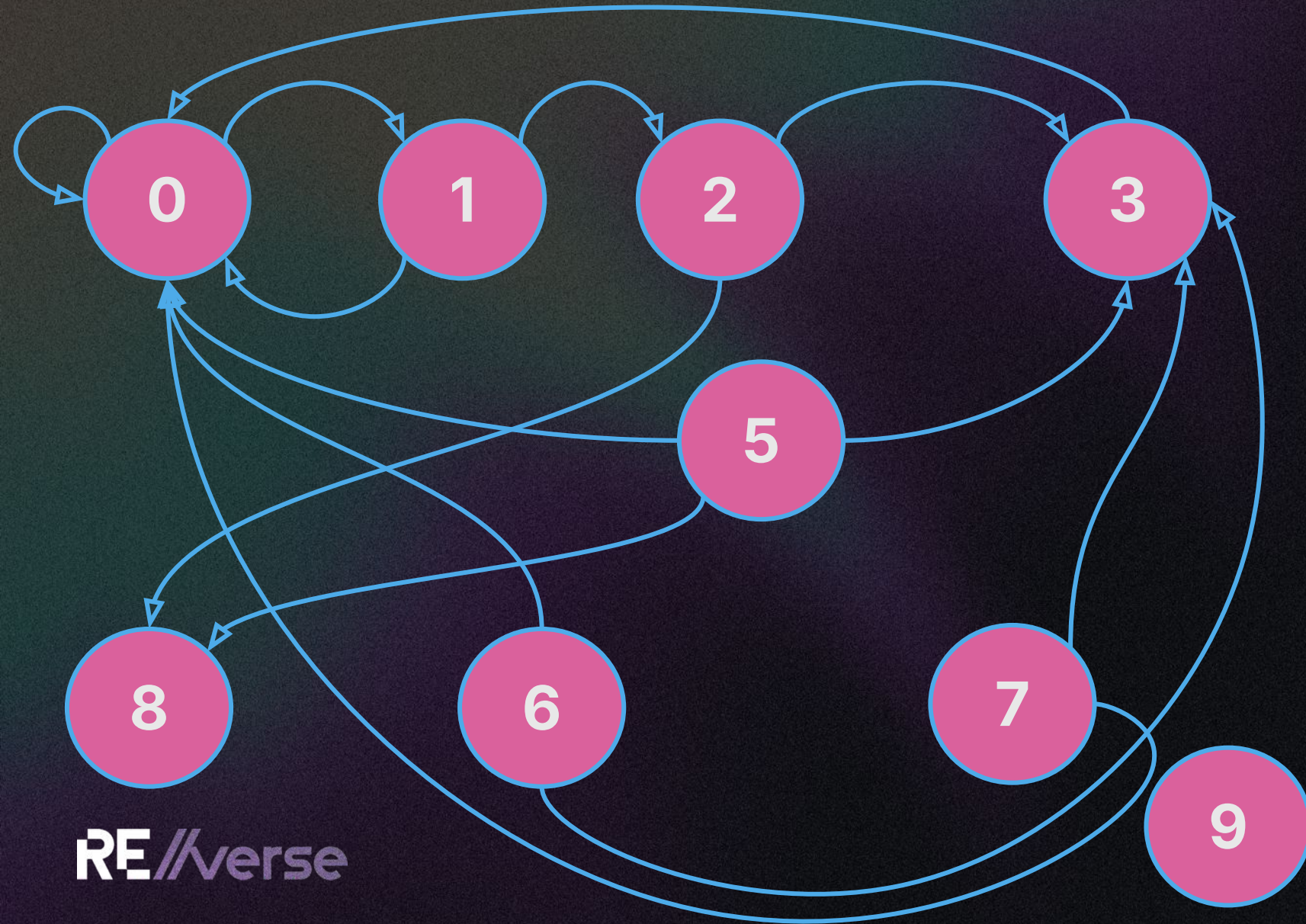
Required steps:

- Mocked USIM state machine
- IRQ support for input delivery
- Connection to SIMurai via PC/SC

```
class USIMPeripheral(LoggingPeripheral):

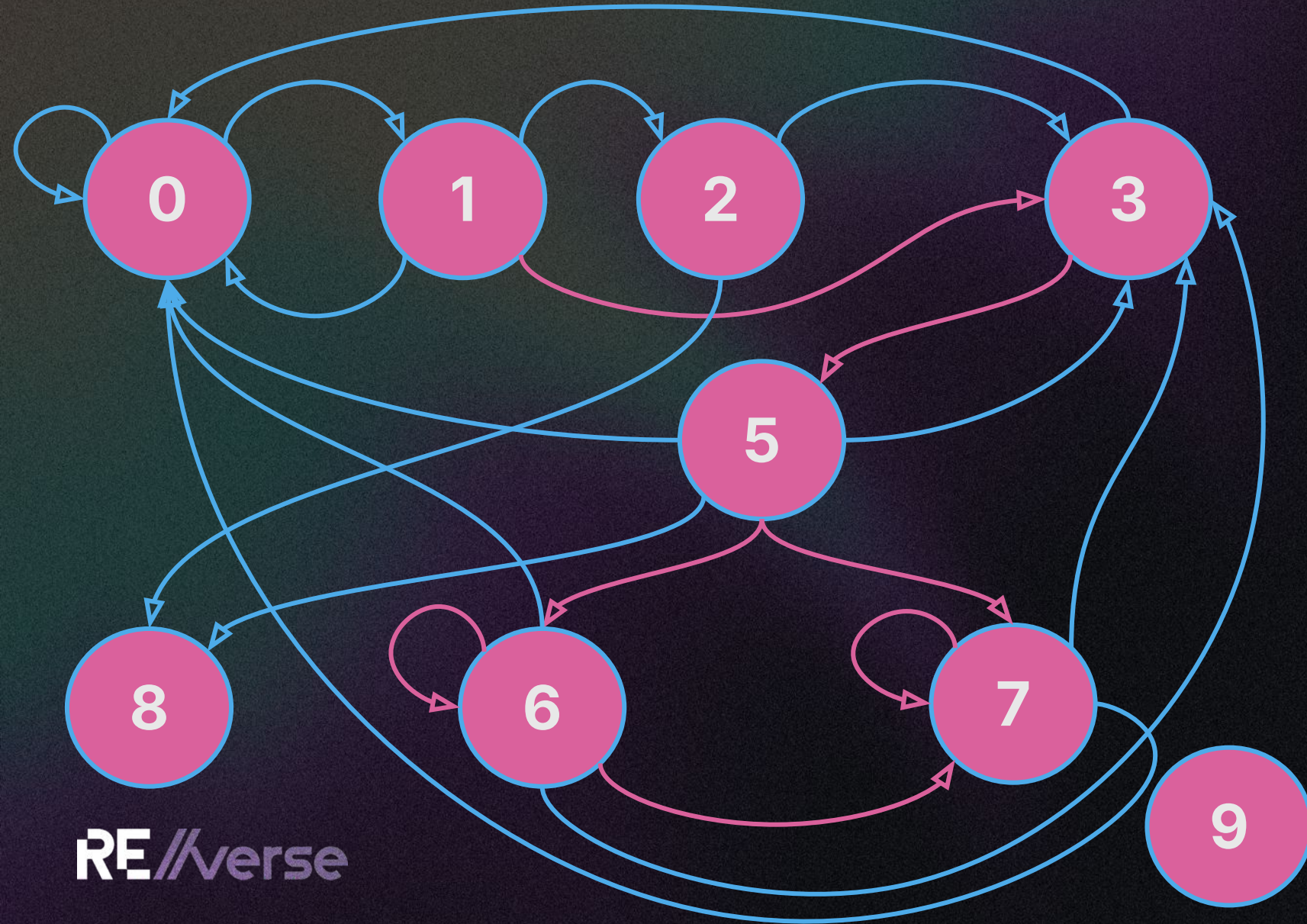
    def hw_read(self, offset, size):
        offset_name = None
        if offset == OFF_DATA_REGISTER:
            offset_name = "DATA"
            value = self.data_register
            self.shall_read_byte = True
        elif offset == OFF_CR0:
            offset_name = "CR0"
            value = self.cr0
        elif offset == OFF_IRQ_PENDING:
            offset_name = "IRQ_Reason"
            value = self.pending_irqs
        # ...
        else:
            self.log_read(value, size, offset_name)
        return value
```

USIM IRQ State Machine: Based on Reverse Engineering



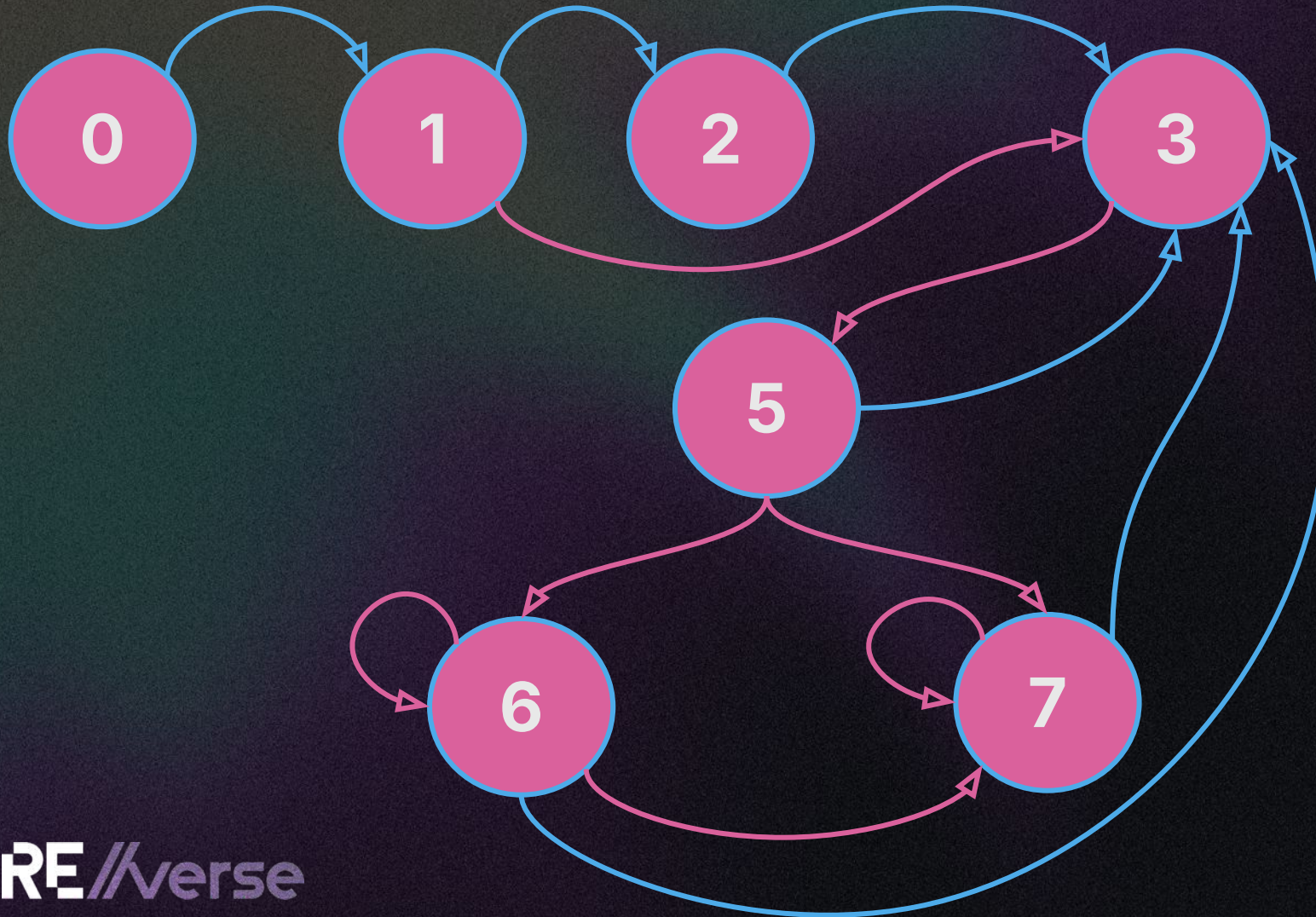
IDX	State Name
0	DRV_IS_NO_SIM
1	WAIT_FOR_ATR
2	DRV_IS_PPS
3	DRV_IS_WAIT_FOR_CMD
4	UNUSED
5	DRV_IS_TX_RX
6	DRV_IS_TX
7	DRV_IS_RX
8	DRV_IS_UNKOWN
9	Abnormal_irq_in_DRV_IS_IDLE

USIM IRQ State Machine: Based on MORE Reverse Eng.



IDX	State Name
0	DRV_IS_NO_SIM
1	WAIT_FOR_ATR
2	DRV_IS_PPS
3	DRV_IS_WAIT_FOR_CMD
4	UNUSED
5	DRV_IS_TX_RX
6	DRV_IS_TX
7	DRV_IS_RX
8	DRV_IS_UNKOWN
9	Abnormal_irq_in_DRV_IS_IDLE

USIM IRQ State Machine, Simplified: No Errors



IDX	State Name
0	DRV_IS_NO_SIM
1	WAIT_FOR_ATR
2	DRV_IS_PPS
3	DRV_IS_WAIT_FOR_CMD
4	UNUSED
5	DRV_IS_TX_RX
6	DRV_IS_TX
7	DRV_IS_RX
8	DRV_IS_UNKOWN
9	Abnormal_irq_in_DRV_IS_IDLE

Fuzzing Setup

SIMurai initializes USIM/USAT

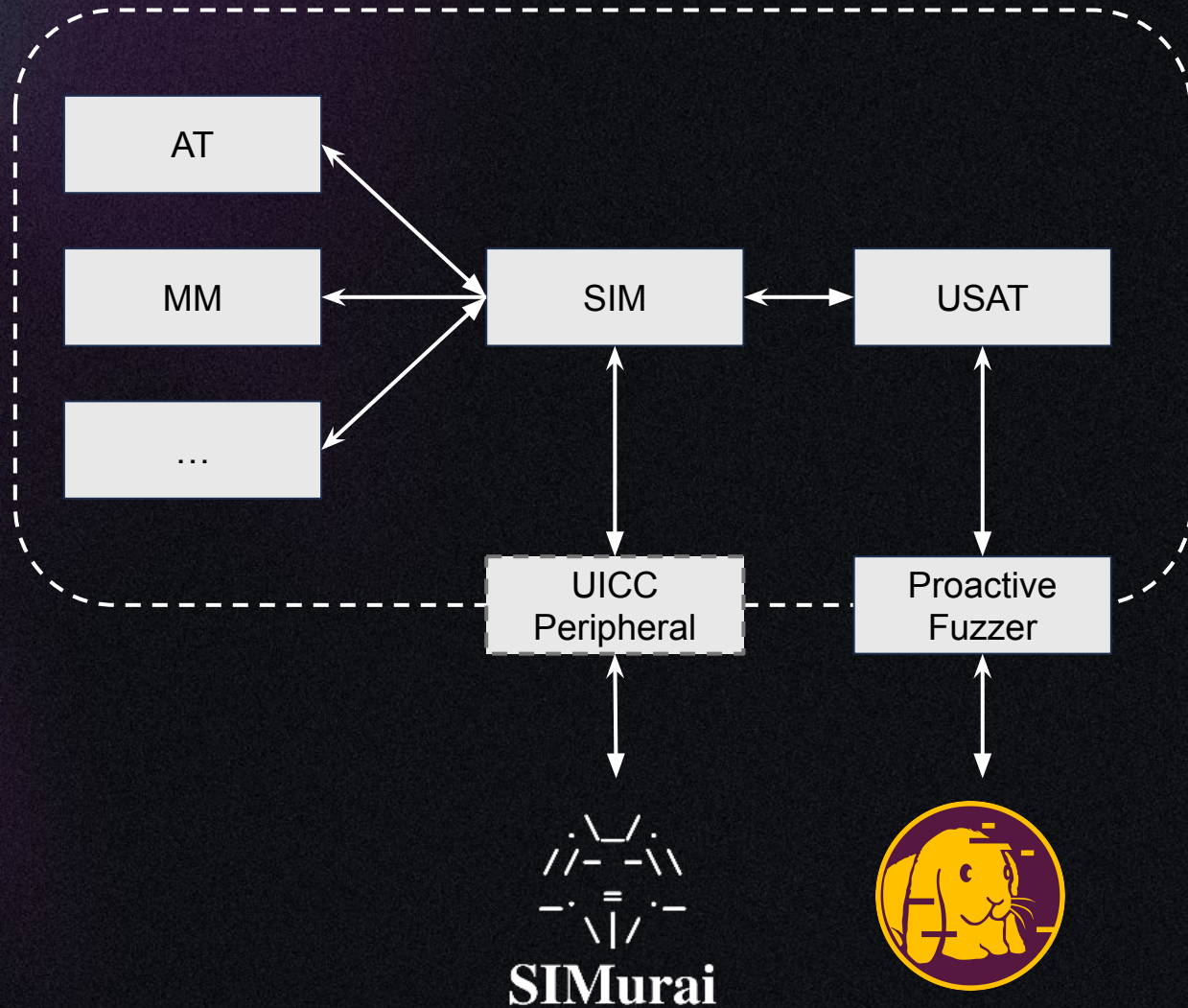
- Effectively: FirmWire with SIM

Snapshot as fuzzing base

- Avoid re-running of lengthy init

Fuzzer directly targets USAT

- Attack surface: proactive cmd parsing



SIM Interface

Vulnerabilities

0

3

1

Interposer

Attack Surface

Baseband

4

2

Fuzz'n'Roll

```
american fuzzy lop ++4.21c {main} (python3) [explore]
process timing -----
    run time : 0 days, 2 hrs, 6 min, 23 sec
    last new find : 0 days, 0 hrs, 0 min, 4 sec
last saved crash : 0 days, 0 hrs, 31 min, 1 sec
last saved hang : 0 days, 0 hrs, 31 min, 9 sec
cycle progress -----
    now processing : 1823*1 (27.2%)
    runs timed out : 0 (0.00%)
stage progress -----
    now trying : splice 1
    stage execs : 9/12 (75.00%)
    total execs : 1.63M
    exec speed : 197.8/sec
fuzzing strategy yields -----
    bit flips : 6/2064, 5/2056, 2/2040
    byte flips : 1/258, 0/250, 1/234
    arithmetics : 7/17.9k, 1/33.8k, 0/31.5k
    known ints : 0/2271, 2/9379, 5/13.0k
    dictionary : 0/0, 0/0, 0/0, 0/0
    havoc/splice : 857/864k, 142/685k
    py/custom/rq : unused, unused, unused, unused
    trim/eff : disabled, 95.35%
strategy: explore ----- state: in progress -----
overall results -----
    cycles done : 9
    corpus count : 6691
    saved crashes : 21
    saved hangs : 11
map coverage -----
    map density : 2.04% / 11.01%
    count coverage : 3.61 bits/tuple
findings in depth -----
    favored items : 295 (4.41%)
    new edges on : 1449 (21.66%)
    total crashes : 125 (21 saved)
    total tmouts : 27 (0 saved)
item geometry -----
    levels : 8
    pending : 2839
    pend fav : 0
    own finds : 1021
    imported : 0
    stability : 67.53%
[cpu000: 50%]
```

Crash Analysis (Emulated): CVE-2023-50806

```

[AFL_USAT] 0x4580024f pal_MsgSendTo(USAT (188)) - PALMsg<0x1f01, USAT (bc) -> USAT_SYNC (bf), 8 bytes>
[USAT] 0x41bdd731: [../../../../../PSS/StackService/USAT/Code/Src/usat_main.c] - [USAT_0]
----- USAT TASK -----

[USAT] 0x41bdc671: [../../../../../PSS/StackService/USAT/Code/Src/usat_main.c] - [USAT_0] 7. USAT <==
SIM_PROACTIVE_CMD_IND [USAT]
[USAT] 0x41bdca53 0b100: [../../../../../PSS/StackService/USAT/Code/Src/usat_main.c] - [USAT_0] usat_State
-> USAT_ACTIVE
[USAT] 0x41bdc97 0b10: [../../../../../PSS/StackService/USAT/Code/Src/usat_main.c] - [USAT_0] Displaying
Received Message: MsgLen = 16
[USAT] 0x41bd89d: [../../../../../PSS/StackService/USAT/Code/Src/usat_ProactiveCmds.c] - [USAT_0]
ProactiveCmdType: PROACTIVE_CMD_SEND_SS
[ ]

[USAT] 0x41bdefe: [../../../../../PSS/StackService/USAT/Code/Src/usat_UsimIntfManagement.c] - [USAT_0] In
usat_CheckCmdSuppInTermProf - Curr Rat - 255, cmd type - 17|
[USAT] 0x41beab9: [../../../../../PSS/StackService/USAT/Code/Src/usat_ProactiveCmdHandler.c] - [USAT_0]
[STK CPLusat_HandleSendSSCmdReqLocal]

[USAT] 0x41beaa2: [../../../../../PSS/StackService/USAT/Code/Src/usat_ProactiveCmdHandler.c] - [USAT_0]
[SEND SS] usat_DecodeSSString
[ERROR] firmware.vendor.shannon.hooks: FATAL ERROR (USAT): from 0x40effd05 [pal_PlatformMisc.c:146 -
Fatal error: PAL_MEM_GUARD_CORRUPTION pal_MemInterface.c Line 895]
```


Fuzzing Results

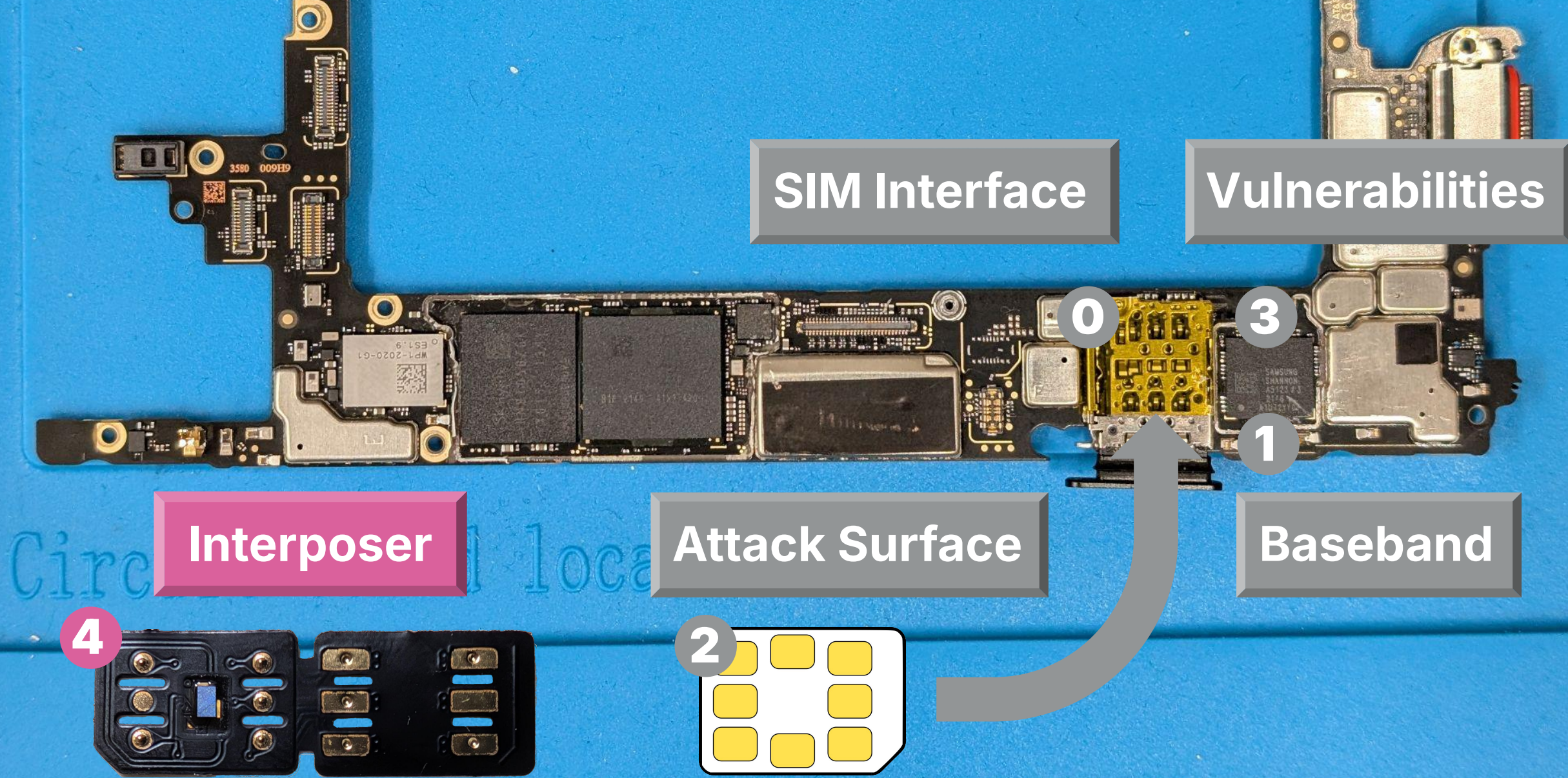
- Multiple crashes, after triaging two different commands:
 - PROACTIVE_SEND_SS
 - PROACTIVE_SEND_SMS
- Both experimentally verified on Google Pixel 6 & 8
 - MODEM_STATE_CRASH! 🎉
- Reported to Google & Samsung:
 - CVE-2023-50806
 - CVE-2024-27209
 - CVE-2024-45184

Is This a Realistic Attack Surface?

- Depends on your threat model:
 - eSIMs
 - Physical attackers
 - OTA Operator updates
 - Vulnerabilities in SIM themselves

But:

What if you would not need a custom SIM at all?



What are interposers?

Machine-in-the-middle hardware implants

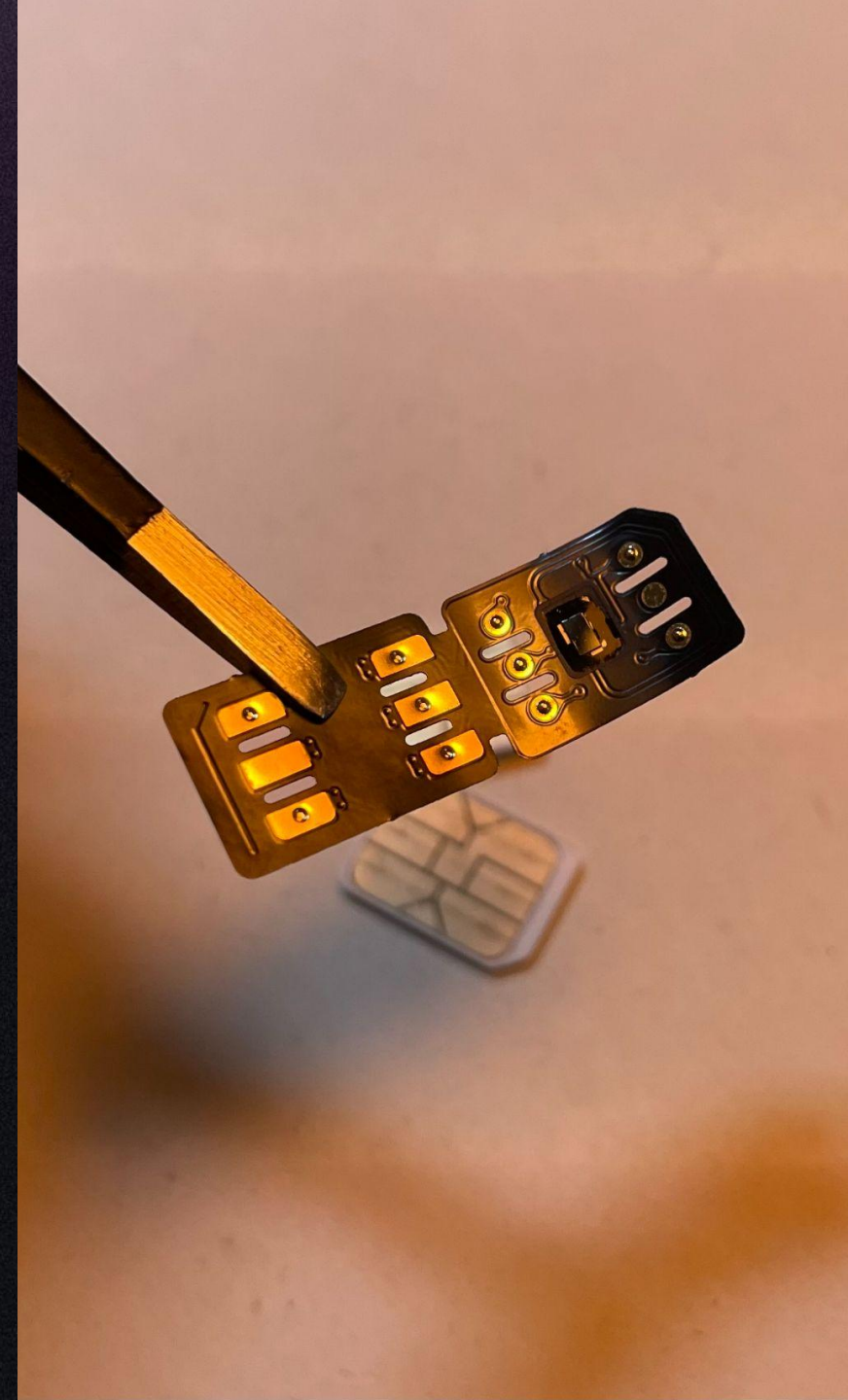
- Tiny chips on flex PCB
- Full control over SIM $\leftrightarrow$ CP communication

Use cases:

- Historic: Extend SIM functionality
- Modern: Carrier lock bypass

Low financial costs:

- Commercially available for less than \$3



Starting point: Custom Firmware

Customisation:

- Web interface
- ICCID & String modification
- Downloadable “firmware file”

Flashing:

- Custom software
- Interposer in card reader

Create Custom Firmware

base on 1.42.715

Set up custom parameters

Set up the title of the skt menu

Custom Title

digit:12

Set up the tips content

An apple a day
keeps the doctor away

digit:36

Set up 20-bit ICCID

74573135531337313371

digit:20

Updated Date:2025-02-09

Set up technical support content

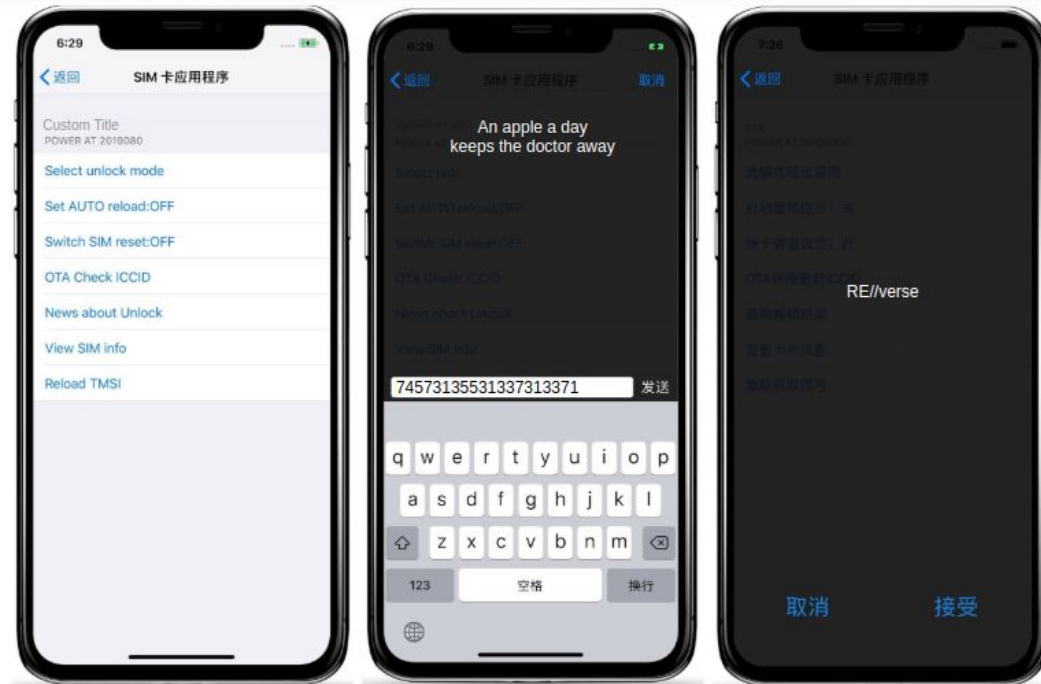
RE//verse

digit:9

Select firmware version you want

v1.42.715

Create



The Firmware (as Downloaded)

[illegible]

Reverse Engineering the Text File

Repeated patterns in sent data:

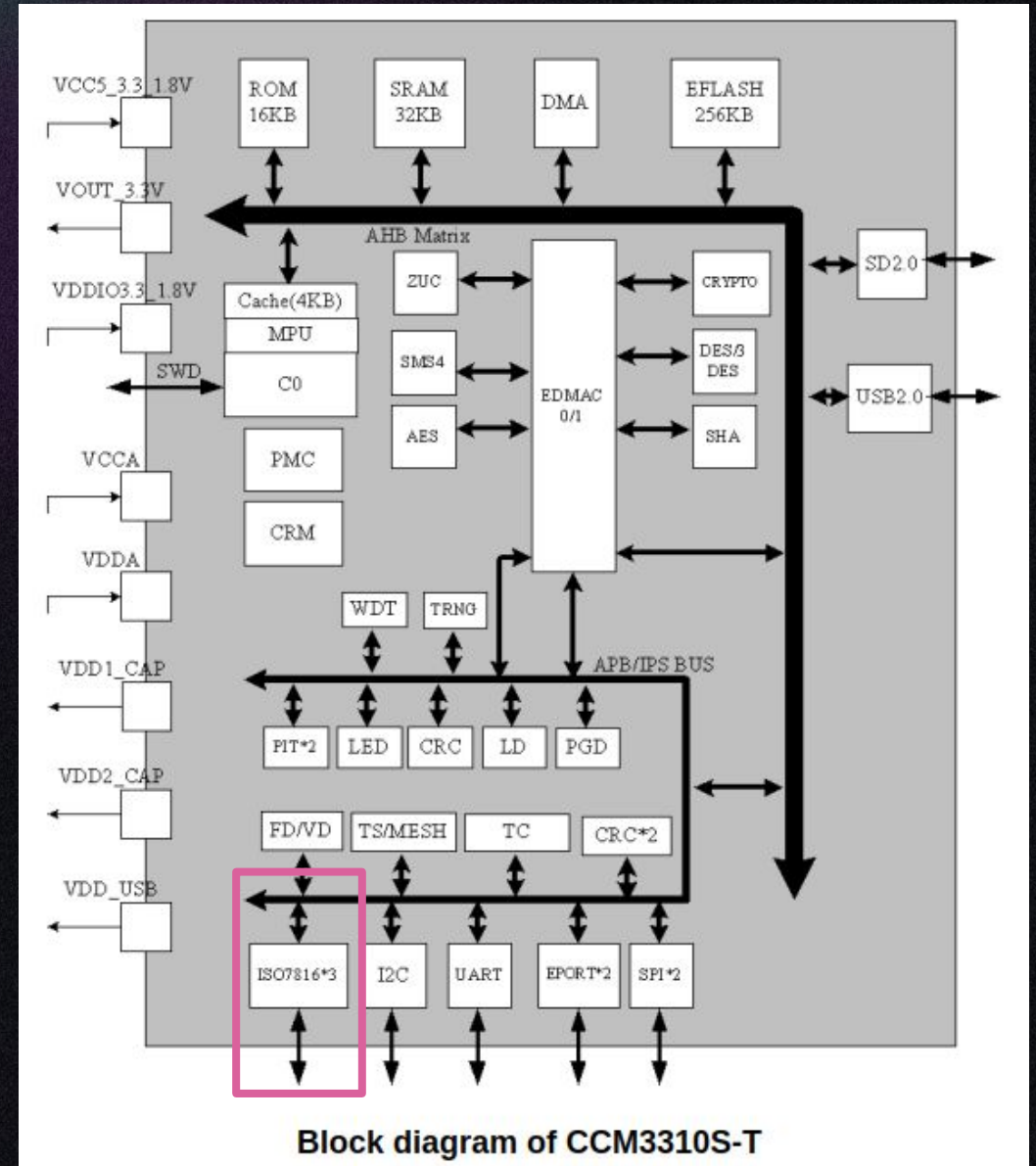
- Hex-encoded APDUs!
- Custom APDU for “flash firmware chunk”
- Created parser to turn into binary

Interesting Strings:

- Chip/Code template: CCM3310S-T SmartCard Demo V1.0
- API/HAL calls (e.g., SmartCard_SendATR/HAL_USI_ATRCallBack
- Menu strings (as shown on phone)

Following the Strings

- Datasheet for chip available
- Includes block diagram with three ISO7816 peripherals
- Information about architecture: RISC-V



Reverse Engineering the Firmware

- Maps perfectly to address 0x00
 - With likely entry point
- Main loop for APDU handling
 - APDU contents stored in global buffers
- ISO7816 peripheral interaction via HAL
 - Actual code not included in firmware

```
void handle_apdu_in(void) {  
    /* ... */  
    apdu_cla = adpu_cla_in_maybe;  
    apdu_ins = apdu_ins_in;  
    apdu_p1 = apdu_p1_in;  
    apdu_p2 = apdu_p2_in;  
    /* ... */  
    if (""[apdu_cla] == 0xff) {  
        periph_do_n_acc_wrap(4);  
        glb_out_sw0_x = 0x6e;  
        glb_out_sw1_x = 0x00;  
        send_sw0();  
        send_sw1();  
    } else {  
        /* ... */  
        apdu_handle_ins_0xab();  
        apdu_dispatch_ins();  
    }  
    return;  
}
```

Modifying the RISC-V Firmware

Goal: Send a malicious payload

- Demonstrate dangers of interposers

Example: PROACTIVE_SEND_SS:

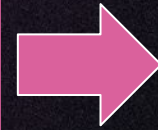
- Heap corruption
- 9 bytes

Approach: Hook FETCH C-APDU

- Sent by UE to get proactive command data

```
void apdu_dispatch_ins(void) {  
    /* ... */  
    if (apdu_ins == 0x10) {  
        handle_term_profile();  
    }  
    else {  
        if (apdu_ins == 0xc2) {  
            handle_envelope();  
        }  
        else {  
            if (apdu_ins == 0x12) {  
                handle_fetch();  
            }  
            else if (apdu_ins == 0x14) {  
                handle_term_response();  
            }  
        }  
    }  
    /* ... */  
}
```

```
void handle_fetch(void) {
    gp = 0x400c60;
    if ((TPDU_len? == '\0') || (do_pa_cmd == '\0')) {
        periph_do_n_acc_wrap(4);
        send_acked_msg_maybe(0x90);
        send_acked_msg_maybe(0);
    }
    else {
        periph_do_n_acc_wrap(4);
        apdu_send_n_bytes(&apdu_ins,1);
        apdu_send_n_bytes(&apdu_out_ptr,apdu_len);
        do_pa_cmd = '\0';
        DAT_004004f5 = 1;
        send_acked_msg_maybe(0x90);
        send_acked_msg_maybe(0);
    }
    return;
}
```



```
void handle_fetch(void) {
    gp = 0x400c60;
    apdu_len = 9;
    memcpy(&apdu_out_ptr,&pl_data,9);
    periph_do_n_acc_wrap(4);
    apdu_send_n_bytes(&apdu_ins,1);
    apdu_send_n_bytes(&apdu_out_ptr,apdu_len);
    ProActive_required = 0;
    DAT_004004f5 = 1;
    send_acked_msg_maybe(0x90);
    send_acked_msg_maybe(0);
    return;
}
```

Flashing & Debugging Infrastructure

Official tooling not ideally for development

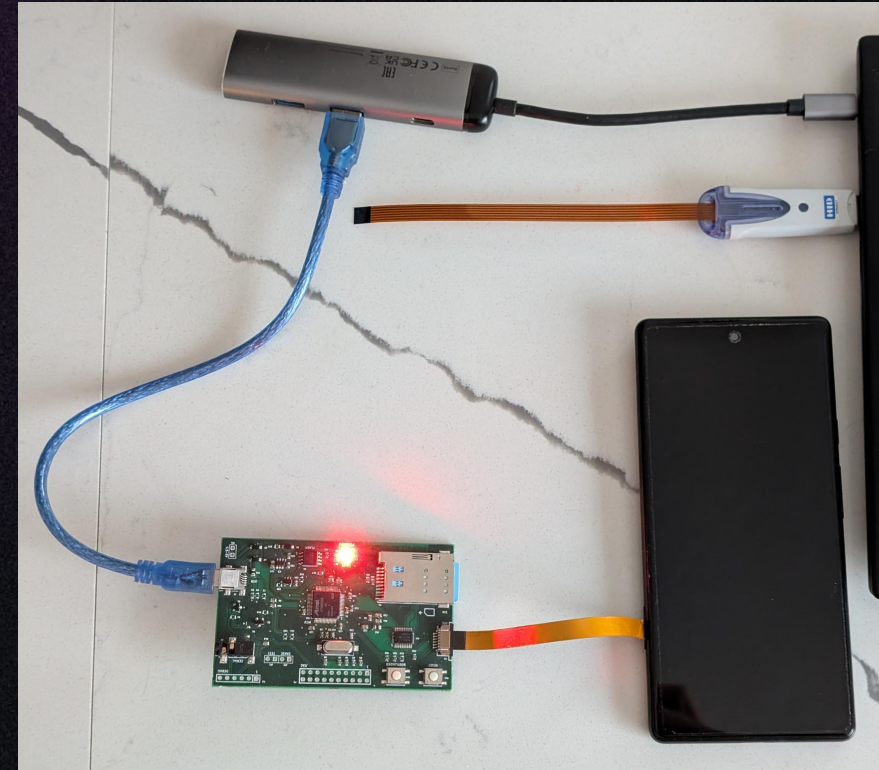
- Manually moving is tedious and fiddly
- Lot's of broken interposers!

SIMTrace2 to the rescue:

- Fast change between phone & card reader
- Tracing capabilities built-in

Replace flashing tool with own python scripts

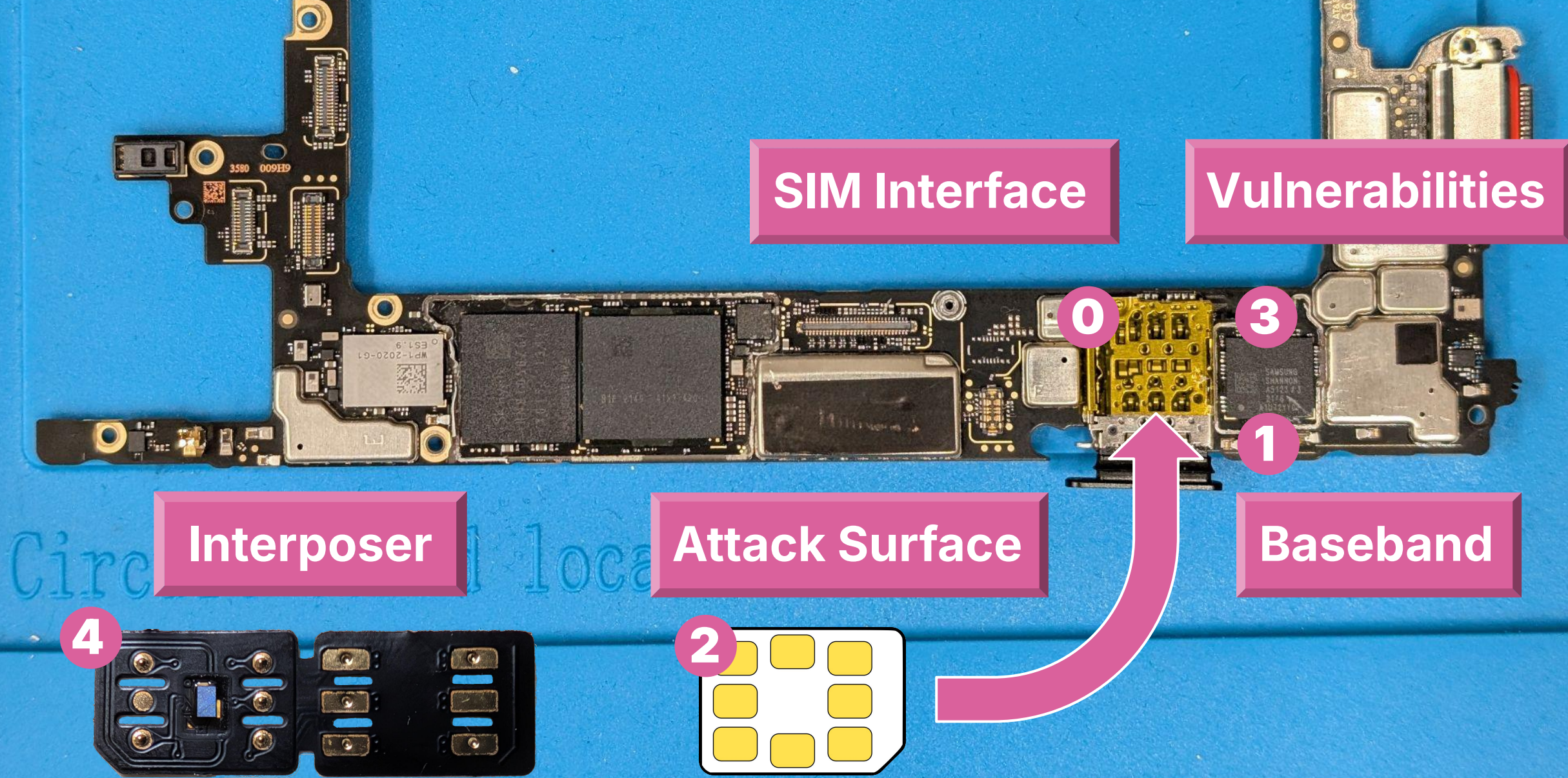
- Faster to iterate & better understanding



RE//verse

Trick #2

Triggering Hack from SIM interposer



Future Work: Defenses

- eSIMs
 - Makes interposers more difficult
- Interposer-based “firewall”
 - Block proactive commands (+ user consent)
- Authenticated SIM communication
 - Secure Channel Protocol (SCP)

Summary

- SIMs are more than a key-value store
 - Instead, tiny computers on their own
- SIM command parsing provide attack surface
 - Main vector: proactive commands
- There is a need for more research SIM tooling
 - SIMurai just the start
- Interposers provide interesting capabilities
 - Especially with firmware updates



Contact us



Talk Slides

REF



SIMurai Paper



SIMurai Github